

Technical Report: YouTube Comment Similarity Analysis using TF-IDF

Gustavo Nascimento Geminiano

July 23, 2026

1 Introduction

This report details the development of a Python script leveraging the YouTube Data API to fetch video comments and evaluate textual similarity between these comments and a user-provided query. The underlying similarity technique relies on TF-IDF vectorization paired with cosine similarity metrics.

2 Objectives

- Automatically extract comments from YouTube videos;
- Preprocess textual data in English using `nltk`;
- Apply the TF-IDF model to convert unstructured text into numerical vector representations;
- Calculate cosine similarity scores between the input query and extracted comments;
- Display the most relevant comments ranked by their similarity scores.

3 Methodology

The program was implemented in Python, utilizing the following core libraries:

- `nltk`: for natural language processing and stopwords filtering;

- **googleapiclient**: for authenticating and interacting with the YouTube Data API;
- **sklearn**: for constructing TF-IDF matrices and computing cosine similarity.

4 Source Code

```
import nltk
from googleapiclient.discovery import build
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import time
import random

nltk.download('stopwords')
from nltk.corpus import stopwords
stopwords_en = stopwords.words('english')

api_key = "YOUR_API_KEY"
link = input("Enter YouTube video link: ")
video_id = link.replace("https://www.youtube.com/watch?v=",
                        ", ")
query = [input("Enter your query: ")]

youtube = build("youtube", "v3", developerKey=api_key)
comments = []
next_page_token = None

print("Downloading comments...")
while True:
    try:
        request = youtube.commentThreads().list(
            part="snippet,replies",
            videoId=video_id,
            maxResults=100,
            pageToken=next_page_token,
            textFormat="plainText"
        )
        response = request.execute()
```

```

        for item in response["items"]:
            comment = item["snippet"]["topLevelComment"]["snippet"]["textDisplay"]
            comments.append(comment)
            if "replies" in item:
                for reply in item["replies"]["comments"]:
                    comments.append(reply["snippet"]["textDisplay"])

        next_page_token = response.get("nextPageToken")
        if not next_page_token:
            break
        time.sleep(random.uniform(0.5, 1.5))
    except Exception as e:
        print(f"Error: {e}")
        time.sleep(5)
        continue

clean_comments = [" ".join([w for w in c.lower().split() if w not in stopwords_en]) for c in comments]
clean_query = [" ".join([w for w in q.lower().split() if w not in stopwords_en]) for q in query]

vectorizer = TfidfVectorizer()
tfidf_docs = vectorizer.fit_transform(clean_comments)
tfidf_query = vectorizer.transform(clean_query)

similarities = cosine_similarity(tfidf_query, tfidf_docs)[0]
sorted_comments = sorted([(c, s) for c, s in zip(comments, similarities) if s > 0], key=lambda x: x[1], reverse=True)

if sorted_comments:
    for i, (comment, score) in enumerate(sorted_comments[:5], 1):
        print(f"{i}. {score:.3f} {comment}")
else:
    print("No relevant comments found.")

```

5 Addenda and Limitations

During testing and execution, it was observed that the script does not always collect 100% of comments from certain videos. This behavior is not caused by flaws in the code, but rather by inherent constraints imposed by the **YouTube Data API**. In specific scenarios, the API fails to return complete page results, particularly on videos featuring exceptionally high comment volumes or deeply nested thread replies. Additionally, API rate limits restrict the number of requests executable within a given time frame. Consequently, the total volume of retrieved comments may be lower than the public counter shown on YouTube. Nonetheless, the sampled subset remains statistically representative for similarity-driven text analyses.

6 Results

The pipeline outputs a ranked list of comments most closely aligned with the search query based on cosine similarity metrics. This mechanism facilitates rapid discovery of relevant answers and audience sentiments across high-volume comment threads.

7 Conclusion

This project demonstrates the practical application of Natural Language Processing (NLP) techniques for unstructured social media content. Potential future extensions include integrating sentiment analysis modules and interactive visual dashboards.